

Module – Motivation for Java Platform Module System (JPMS)

The Need for Modularity

Key Points

- As Java applications grew larger, **managing dependencies and visibility** became complex.
- The traditional JAR-based system had **no explicit boundaries** — everything was public and accessible.
- Developers struggled with:
 - Version conflicts (“JAR Hell”).
 - Lack of encapsulation between packages.
 - Hard-to-manage runtime dependencies.
- JPMS was introduced in Java 9 to bring **true modularization** to the platform and user code.

Code Snippet

```
// Traditional Java
// Any class from any JAR can access public APIs of another JAR
import com.example.internal.Helper; // unwanted dependency but compiles
fine
```

Goals of JPMS

Key Points

- The Java Platform Module System (JPMS) aims to:
 - **Encapsulate code** and expose only intended APIs.
 - **Define clear dependencies** between components.
 - **Eliminate classpath conflicts** via module paths.
 - **Support reliable configuration** — the JVM knows which modules are needed at startup.
 - **Enable scalable deployment** and runtime optimization.
- JPMS also modularized the **entire Java SE platform**.

Code Snippet

```
// Example module declaration
module com.learnato.billing {
    requires com.learnato.payments;
    exports com.learnato.billing.api;
}
```

Classpath vs Module Path

Key Points

- **Classpath** loads all classes into a flat namespace — no visibility control.
- **Module Path** enforces strong encapsulation and explicit dependency declarations.
- Modules specify what they **require** and what they **export**.
- The JVM verifies dependency consistency before startup.

Comparison Example

Feature	Classpath	Module Path
Dependency Declaration	Implicit	Explicit (<code>requires</code>)
Encapsulation	None	Strong (controlled via <code>exports</code>)
Version Control	Manual	Enforced by module descriptors

Code Snippet

```
# Running on classpath
java -cp . com.learnato.Main

# Running on module path
java --module-path mods -m com.learnato/com.learnato.Main
```

Modularity in the Java Platform

Key Points

- JPMS divides the Java SE platform into **standard modules** like:
 - `java.base` (core APIs)
 - `java.sql` (database access)
 - `java.xml` (XML processing)
 - `java.desktop` (GUI)
- `java.base` is **implicitly required** by all modules.
- Applications can include only the modules they need, improving performance and footprint.

Code Snippet

```
module my.app {  
    requires java.sql;  
    requires java.xml;  
}
```

Benefits of the Module System

Key Points

- **Strong Encapsulation** – only exported packages are visible to other modules.
- **Reliable Configuration** – explicit `requires` statements ensure missing modules are detected early.
- **Improved Security** – internal packages stay hidden from external access.
- **Smaller Runtime Images** – with tools like `jlink`, custom JREs can include only required modules.
- **Better Maintenance** – clear structure for large multi-team projects.

Code Snippet

```
module com.learnato.analytics {  
    requires com.learnato.data;  
    exports com.learnato.analytics.api;  
}
```

How JPMS Fits into the Build and Runtime Process

Key Points

- Modules introduce a new **module descriptor file**: `module-info.java`.
- Compilation uses `--module-source-path` to maintain modular structure.
- At runtime, modules are resolved through `--module-path`.
- IDEs like IntelliJ and Eclipse have built-in support for modular projects.

Build Example

```
# Compiling modules
javac -d mods --module-source-path src $(find src -name "*.java")

# Running a module
java --module-path mods -m com.learnato.main/com.learnato.Main
```

Backward Compatibility and Migration

Key Points

- JPMS is **backward-compatible** with traditional JAR-based projects.
- Legacy JARs placed on the module path become **automatic modules**.
- Automatic modules derive names from JAR filenames and export all packages.
- Gradual migration is possible — existing applications can move to JPMS incrementally.

Code Snippet

```
# Example of automatic module naming  
my-library.jar → module name: my.library
```

Certification Tips

Key Points

- `java.base` is always required — no need to declare it.
- Each module has its own **module descriptor** (`module-info.java`).
- The `exports` keyword exposes specific packages; others remain internal.
- The `requires` keyword defines dependencies.
- Automatic modules bridge classpath and module path for migration.
- Expect questions comparing **classpath vs module path**, and identifying **module visibility rules**.

Code Snippet

```
module com.learnato.sales {  
    requires com.learnato.products;  
    exports com.learnato.sales.api;  
}
```

Module – Module Declarations and module-info.java Structure

Introduction to module-info.java

Key Points

- Every Java module is defined by a special file named `module-info.java`.
- It acts as the **entry point and descriptor** of a module.
- Located in the **root of the module's source folder**.
- Specifies dependencies, exported packages, services, and accessibility rules.

Code Snippet

```
module com.learnato.billing {  
    requires com.learnato.payments;  
    exports com.learnato.billing.api;  
}
```

Exercise 01 – ModuleIntro.java

Create a `module-info.java` file for a module `com.learnato.store` that depends on `com.learnato.inventory`.

Syntax and Keywords

Key Points

- The general structure of a module descriptor includes:
 - `module <name>` → defines the module.
 - `requires <module>` → declares dependency.
 - `exports <package>` → exposes public packages.
 - `opens <package>` → allows runtime reflection access.
 - `uses` and `provides` → support service-based architectures.

Code Snippet

```
module com.learnato.service {  
    requires com.learnato.utils;  
    exports com.learnato.service.api;  
    opens com.learnato.service.internal;  
}
```

Exercise 02 – SyntaxPractice.java

Write a module descriptor using `requires`, `exports`, and `opens` for reflection.

requires and requires transitive

Key Points

- `requires` indicates that the module depends on another.
- `requires transitive` exposes the dependency to modules that depend on you.
- Useful for layered architectures where multiple modules depend on a shared core.

Code Snippet

```
module com.learnato.ui {  
    requires transitive com.learnato.core;  
}  
module com.learnato.app {  
    requires com.learnato.ui; // gains access to com.learnato.core  
    automatically  
}
```

Exercise 03 – TransitiveDependency.java

Define three modules (A, B, C) where A requires transitive B, and C requires A. Verify that C can access B's exports.

exports and opens

Key Points

- `exports` makes a package's public types visible to other modules.
- Only explicitly exported packages are accessible.
- `opens` allows runtime reflection (used by frameworks like Hibernate, Jackson).
- `opens` does not grant compile-time visibility — only runtime.

Code Snippet

```
module com.learnato.model {  
    exports com.learnato.model.api;  
    opens com.learnato.model.entities to com.fasterxml.jackson.databind;  
}
```

Exercise 04 – ExportOpenDemo.java

Export one package for normal access and open another for reflection. Note which one is visible at compile time.

uses and provides (Service Loader Pattern)

Key Points

- `uses` declares a service interface your module consumes.
- `provides ... with ...` declares a service implementation your module supplies.
- Together they support the **Service Loader** mechanism for pluggable architectures.

Code Snippet

```
// In com.learnato.payments
module com.learnato.payments {
    exports com.learnato.payments.api;
}

// In com.learnato.payments.impl
module com.learnato.payments.impl {
    requires com.learnato.payments;
    provides com.learnato.payments.api.PaymentService
        with com.learnato.payments.impl.CardPaymentService;
}
```

Exercise 05 – ServiceProvider.java

Create an interface and its implementation, then define `uses` and `provides` in module descriptors.

requires static and open modules

Key Points

- `requires static` means dependency is only needed at **compile time**, not runtime.
- Useful for optional features like annotation processors or testing libraries.
- `open module` grants reflective access to all packages — often used for test or utility modules.

Code Snippet

```
open module com.learnato.testing {  
    requires static org.junit.jupiter.api;  
}
```

Exercise 06 – StaticDependency.java

Create an `open module` for testing that depends statically on JUnit.

module-info Compilation and Location

Key Points

- `module-info.java` must reside in the root of the module's source directory.
- During compilation, it produces a **`module-info.class`** file inside the output directory.
- The JVM uses this descriptor to verify module boundaries at runtime.

Code Snippet

```
src/  
└─ com.learnato.billing/  
    └─ module-info.java  
    └─ com.learnato.billing/api/BillService.java
```

Exercise 07 – ModuleLayout.java

Sketch the directory layout of a valid Java module with one exported and one non-exported package.

Certification Tips

Key Points

- Every module has exactly one `module-info.java`.
- `exports` controls compile-time visibility; `opens` controls runtime reflection.
- `requires transitive` propagates dependencies; `requires static` is compile-time only.
- `open module` opens all packages by default.
- `provides/uses` enable service loading via `ServiceLoader`.
- Expect questions involving visibility, dependency resolution, and invalid descriptors.

Code Snippet

```
module com.learnato.analytics {  
    requires transitive com.learnato.data;  
    exports com.learnato.analytics.api;  
}
```

Exercise 10 – CertificationPractice.java

Predict which modules can access `com.learnato.data.api` when using `requires transitive`.

Module – Compiling and Running Modular Applications

From Source to Module

Key Points

- Each module must have its own `module-info.java` descriptor and organized folder structure.
- The compiler (`javac`) and runtime (`java`) commands must both know the **module path**.
- The `--module-source-path` flag tells the compiler where multiple modules are located.
- Modules are compiled into separate directories under the `mods` folder.

Code Snippet

```
javac -d mods --module-source-path src $(find src -name "*.java")
```

Exercise 01 – MultiModuleCompile.java

Create two modules `com.learnato.core` and `com.learnato.app`, and compile them together using `--module-source-path`.

Module Directory Layout

Key Points

- The **module name** corresponds to the folder under the source directory.
- Each module contains its own `module-info.java` and package hierarchy.
- Output structure after compilation mirrors the source module layout.

Code Snippet

```
src/  
├── com.learnato.core/  
│   ├── module-info.java  
│   └── com.learnato.core/CoreService.java  
└── com.learnato.app/  
    ├── module-info.java  
    └── com.learnato.app/MainApp.java
```

Exercise 02 – VerifyLayout.java

Sketch a folder layout for a project with three modules: `core`, `service`, and `app`, each with one package.

Running a Modular Application

Key Points

- Use `java --module-path` to specify where compiled modules are located.
- Use `-m <module>/<main-class>` to run a specific module.
- The JVM verifies module dependencies before launching the main class.

Code Snippet

```
java --module-path mods -m com.learnato.app/com.learnato.app.MainApp
```

Exercise 03 – RunModule.java

Run the `com.learnato.app` module which depends on `com.learnato.core`. Verify that missing modules trigger errors.